Machine Coding Round: Strategy & Setup

7.1 - Machine Coding Round: Strategy & Setup

What is the Machine Coding Round?

The machine coding round is a timed (usually 45-90 minutes) interview where you build a working React component or small application from scratch. The interviewer gives you a problem statement, and you code it live while they watch.

This round tests more than just syntax. It reveals how you think, how you structure code, and whether you can ship something functional under pressure.

What Interviewers Evaluate

What they watch for	What impresses them	What raises re
Component structure	Small, focused components	One massive c
State management	State close to where it's used	Everything in a

Edge cases	Empty states, loading, error handling	Only handles t
Code organization	Separation of concerns, reusable pieces	Copy-pasted l
Communication	Thinking out loud, explaining tradeoffs	Coding silently

The 5-Minute Planning Phase

Before writing any code, spend 5 minutes planning. This is what separates strong candidates from those who rush into code and get stuck halfway.

Step 1: Clarify requirements. Ask the interviewer: "Should it support X? What happens when Y? Is Z in scope?" This shows you think before coding.

Step 2: Identify the state. What data needs to be tracked? What changes? Write it down mentally: "I need a list of items, an input value, and a filter."

Step 3: Sketch the component tree. Which components do I need? What's the parent, what are the children? What props flow where?

Step 4: Pick your approach. useState vs useReducer? Controlled vs uncontrolled inputs? Single component or multiple?

Step 5: Start with the simplest working version. Get the core feature working first, then add polish (validation, edge cases, styling) if time permits.

The Template

Every machine coding problem in this module follows this structure:

Problem Brief: What you're asked to build, described the way an interviewer would say it.

Requirements: The specific features expected.

What This Tests: The React concepts the interviewer is evaluating.

Complete Solution: Production-quality code you can study and practice.

Key Concepts Demonstrated: A list of the patterns and techniques used.

Extension Ideas: How to extend the solution if you finish early (bonus points).

Tips for Success

Talk while you code. Narrate your decisions: "I'm going to start with the state. I need an array of todos and an input string. I'll use useState for both since they're simple."

Start ugly, refactor later. Get it working in one component first. Extract into smaller components once the logic is solid. Interviewers would rather see a working ugly solution than a perfectly structured broken one.

Handle the empty state. If the list is empty, show a message. If the input is blank, disable the button. These small details show maturity.

Don't over-engineer. If the problem asks for a counter, don't build a Redux store. Match the complexity of your solution to the complexity of the problem.

Know when to stop. If the core feature works with 15 minutes left, add error handling, edge cases, or accessibility. Don't start a major refactor in the last 5 minutes.

QUICK CHECK — no marks, just for you.

1. The interviewer finishes reading out the problem statement. According to the lesson, what should you do in the first five minutes?

- ○ Set up state management and folder structure so the architecture is locked in before any feature work

- [ ] Write the full component tree as empty files first, then fill each one in top-down
- [ ] Plan: clarify requirements, identify the state, sketch the component tree, and pick an approach before writing code

Check answer

2. You are 30 minutes into a 45-minute round. The core feature works but it all lives in one messy component. Based on the lesson, what does the interviewer most want to see?

- [ ] Leave it as-is and spend the rest of the time on styling, since interviewers judge the visual result
- [ ] A working but ugly solution beats a well-structured one that does not run, so finish the feature and refactor only if time allows
- [ ] Rewrite the state layer with useReducer to show you know advanced patterns
- [ ] Stop and split it into clean components now, since component structure is what is actually being graded

Check answer

3. On the state-management criterion, which candidate behaviour does the lesson list as a red flag rather than something that impresses?

- [ ] Choosing useState for simple values and useReducer for more complex transitions
- [ ] Hoisting everything into a single useState at the top of the app
- [ ] Keeping state close to where it is actually used
- [ ] Rendering an empty-state message when the list has no items

Previous
Behavioral Questions: What Interviewers Really Want

Next
Build: Counter with useReducer

LEARN MORE

ARTICLES & DOCS

- Thinking in React (how to approach)
  react.dev
- Tutorial: Tic-Tac-Toe (practice)
  react.dev

VIDEOS



- React machine coding interview prep
  YouTube



- React machine coding round questions
  YouTube



- Machine coding — a worked example
  YouTube

Hand-picked references for this lesson.